Bitbucket Read-Your-Writes Consistency & LSN Routing

Source: Arpit Bhayani Dissection • Bitbucket Database Engineering
Focus: Master Overload, Replication Lag & WAL LSN Mechanics

1. The Primary Saturation Bottleneck

In relational master-replica database architectures (such as PostgreSQL), scaling read traffic horizontally by provisioning read replicas is the standard industry pattern. Writes execute on the Primary (Master) database and stream asynchronously to multiple Read Replicas via the Write-Ahead Log (WAL).

The Problem: Asynchronous replication introduces a finite replication delay (typically 20–200 milliseconds). If an application routes reads indiscriminately to replicas, users encounter the **Read-Your-Writes (RYW) Anomaly**: a developer merges a pull request or posts a comment, immediately refreshes the page, and the page renders stale data without their changes. To protect user experience, engineers often route all sensitive reads to the Primary, quickly saturating the Primary CPU and connection pools.

2. Anatomy of the Read-Your-Writes Failure Mode



3. The LSN Solution: PostgreSQL WAL Mechanics

Bitbucket rejected blunt heuristics (such as pinning all users to Primary for 10 seconds after a write) and instead leveraged the physical ordering primitives of PostgreSQL: **Log Sequence Numbers (LSNs)**.

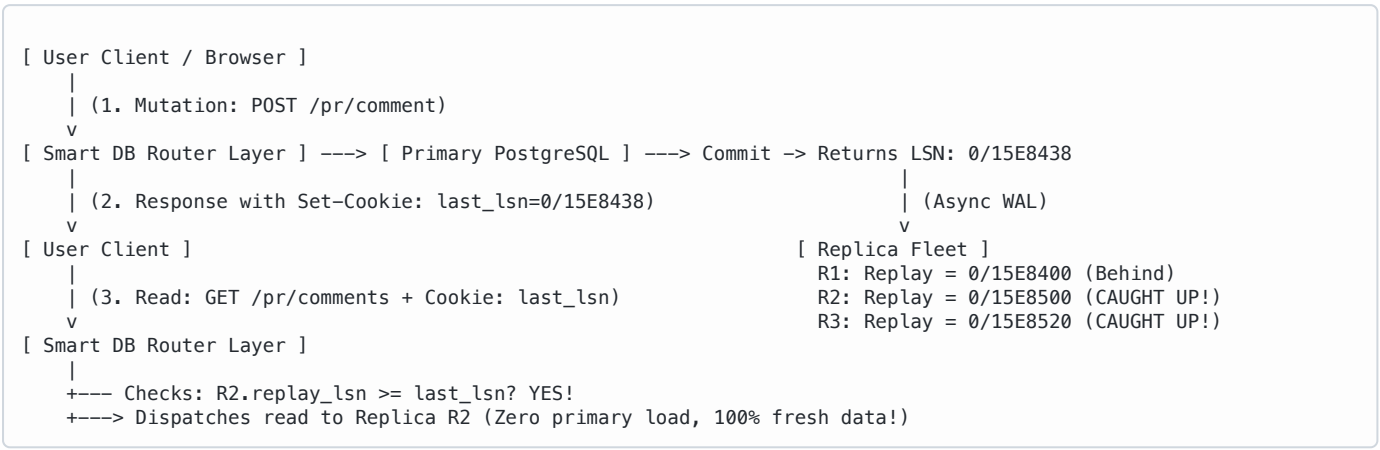
- Every transaction committed in PostgreSQL appends data to the Write-Ahead Log at a monotonically increasing 64-bit integer byte position: the **LSN** (e.g. 16/B374D848).
- When a user writes to the Primary, the application captures `pg_current_wal_lsn()` and stores it in the user's session cookie or JWT token.
- When the user subsequently issues a read query, the database router checks `pg_last_wal_replay_lsn()` across the replica fleet.

> **Mathematical Invariant:** If $ ext{LSN}_{ ext{replay}}^{(R)} \ge ext{LSN}_{ ext{user_write}}$, Replica R is mathematically guaranteed to contain the user's latest committed changes. The read query is dispatched safely to Replica R with zero possibility of stale data.

Dynamic Replica Routing Architecture & Telemetry

Source: Arpit Bhayani Dissection • Bitbucket Database Engineering
Focus: LSN Session Tokens, Replica Telemetry & Fallback Loops

4. End-to-End Dynamic Replica Routing Topology



5. Engine Queries & Synchronization Logic

The routing layer executes lightweight built-in functions without table locking:

- **On Write Completion (Primary):** `SELECT pg_current_wal_lsn();` → Captures exact transaction boundary.
- **Replica Telemetry (Replica Health Poller):** `SELECT pg_last_wal_replay_lsn();` → Tracks latest replayed byte offset.
- **Replication Offset Comparison:** `SELECT pg_wal_lsn_diff(pg_last_wal_replay_lsn(), '0/15E8438'::pg_lsn);` Returns positive if the replica is ahead of the target LSN, negative if lagging.

6. Edge Cases & Fallback Policies

Scenario	System Behavior	Operational Guardrail
All Replicas Lagging	Micro-wait up to 50ms for closest replica to catch up; fallback to Primary if exceeded.	Guarantees strict causal consistency under heavy write bursts.
Token Expiry	Client LSN tokens expire after 15–30 seconds.	Once all replicas are far past the write, user returns to standard round-robin balancing.
Replica Network Partition	Router marks partitioned node offline based on stalled heartbeat LSN.	Prevents routing reads to stranded replicas.

Production Impact & Architectural Takeaways

Source: Arpit Bhayani Dissection • Bitbucket Database Engineering
Focus: 50% Load Reduction, Causal Guarantees & Systems Principles

7. Production Impact: 50% Primary Load Reduction

By moving from blunt heuristics to LSN-guided routing, Bitbucket achieved dramatic operational improvements:

Metric / Dimension	Before (Primary Pinned Reads)	After (LSN Guided Routing)
Primary Database Query Load	100% saturation during peak collaboration hours	**50% reduction in query QPS**
Read-Your-Writes Inconsistencies	Recurring user complaints of missing comments/commits	**Zero reported causal consistency anomalies**
Replica Fleet Utilization	Underutilized; rejected for sensitive user flows	**Evenly distributed load across read pool**
Database Latency Spikes	Frequent p99 spikes due to Primary connection contention	Predictable sub-10ms response times platform-wide

8. First-Principles System Design Takeaways

- **Storage Engine Primitives Beat Application Timers:** Arbitrary sleep timers and fixed sticky windows are fragile and waste resources. Relational databases already track write sequence with mathematical precision via WAL LSNs (Postgres) and GTIDs (MySQL).
- **Causal Consistency at Zero Coordination Cost:** Enforcing full distributed serializability is expensive and complex. In contrast, *Read-Your-Writes (Causal) consistency* can be achieved cheaply by passing progress tokens to clients.
- **Stateless Routing via Client Context:** Embedding the target LSN in a client cookie or session token avoids centralized routing state machines, allowing stateless edge proxies to make instantaneous routing decisions.
- **Unlocking Horizontal Scale:** Once read-your-writes guarantees are decoupled from the Primary database, the read replica fleet can scale horizontally to dozens of instances without risking stale user reads.